

Fore-and-Back for Changepoint Analysis

Vittorio Maniezzo¹[0000–0002–1220–1235]

University of Bologna, Cesena, Italy
vittorio.maniezzo@unibo.it
<https://www.unibo.it/sitoweb/vittorio.maniezzo/en>

Abstract. This paper introduces a novel fore–n–back matheuristic for offline changepoint detection in univariate time series, combining the modeling rigor of mathematical programming with the efficiency of heuristic search. Classical exact approaches to optimal partitioning, such as dynamic programming, offer strong optimality guarantees but typically exhibit quadratic time and memory requirements in the sequence length, which can limit their applicability to long series or resource-constrained settings. The proposed method embeds an optimization-based formulation within an iterative forward–backward search scheme that rapidly constructs high-quality feasible segmentations and incrementally improves them as additional computational resources become available. This design yields an anytime algorithm that preserves the structural advantages of exact models—such as extensibility and constraint handling—while avoiding exhaustive exploration. Computational experiments show that the matheuristic consistently attains near-optimal solutions at a fraction of the computational cost of standard exact methods, providing a scalable and flexible alternative for optimization-based changepoint detection in practice.

Keywords: Changepoint analysis · matheuristics · Fore-and-Back.

1 Introduction

Changepoint detection (CPD) is the problem of identifying locations in a time series where the underlying data-generating process changes abruptly. Such changes may correspond to shifts in mean, variance, trend, or more complex structural properties, and their accurate detection is central to a wide range of applications, including industrial process monitoring, finance, bioinformatics, network security, and environmental analysis. As modern sensing and logging technologies generate increasingly long and high-frequency time series, there is a growing demand for CPD methods that are not only statistically sound but also computationally scalable and adaptable to practical constraints.

A common and widely studied formulation of CPD is the optimal partitioning problem, in which the time series is segmented into contiguous intervals so as to minimize a global objective composed of within-segment costs and a penalty on the number of changepoints. This framework encompasses many classical likelihood-based and least-squares models and admits exact solution

methods with strong optimality guarantees. Dynamic programming (DP) approaches constitute the canonical exact solution technique for this problem and, when segment costs can be computed efficiently, achieve quadratic time and memory complexity in the length of the series. Despite important advances such as pruning strategies and average-case improvements, these methods may still become computationally demanding for very long sequences, complex segment models, or when additional structural constraints are imposed.

In parallel to algorithmic developments, mathematical programming formulations—typically based on mixed-integer linear or quadratic models—have been proposed to address CPD in a unified optimization framework. These approaches offer several advantages, including modeling flexibility, the ability to incorporate global constraints across segments, and conceptual clarity. However, their direct solution via general-purpose solvers often requires exploring a large combinatorial search space, which can limit their practical applicability, especially in low-latency or resource-constrained environments. As a result, there remains a gap between the expressive power of optimization-based CPD models and the computational efficiency demanded by large-scale or time-critical applications.

Matheuristics, which integrate mathematical programming techniques within heuristic or metaheuristic frameworks, provide a promising avenue for bridging this gap. By leveraging the structural strength of exact models while selectively relaxing exhaustive search, matheuristics aim to deliver high-quality solutions with significantly reduced computational effort. In the context of CPD, however, relatively few approaches explicitly exploit this paradigm, and most existing methods fall either on the side of exact optimization or on purely algorithmic heuristics.

This paper proposes a novel matheuristic for univariate changepoint detection based on a **fore–n–back** search strategy. The method starts from a rapidly generated feasible segmentation and alternates between forward construction and backward refinement phases, guided by an underlying mathematical programming formulation. This design enables the algorithm to quickly identify promising regions of the solution space while retaining a global view of the segmentation problem. Importantly, the approach is **anytime** in nature: it produces a valid solution almost immediately and progressively improves solution quality as additional computational resources become available.

The proposed matheuristic preserves key advantages of optimization-based CPD models. In particular, it naturally supports the inclusion of global constraints on the segmentation, such as bounds on the number of segments or minimum segment lengths, which are difficult to accommodate within classical dynamic programming frameworks. At the same time, by avoiding a full enumeration of candidate segmentations, the method circumvents the computational burden typically associated with exact solvers.

Extensive computational experiments demonstrate that the proposed approach consistently yields near-optimal solutions while requiring only a fraction of the computational effort of standard exact methods. These results suggest that the *fore–n–back* matheuristic provides an effective compromise between statis-

tical accuracy and runtime efficiency, making it well suited for large-scale and resource-constrained CPD applications.

The remainder of the paper is organized as follows. Section 2 reviews related work on changepoint detection and optimization-based approaches. Section 3 introduces the mathematical programming formulation underlying the proposed method. Section 4 details the fore–n–back heuristic and its algorithmic components. Section 5 presents computational results, and Section 6 concludes with directions for future research.

2 Changepoint analysis

Changepoint detection has been extensively studied across statistics, machine learning, and operations research, leading to a rich body of methodological literature. Early approaches focused on hypothesis testing and likelihood ratio methods applied sequentially or retrospectively, often with limited scalability or weak global optimality guarantees. More recent work has converged on segmentation-based formulations, in which the time series is partitioned into contiguous segments by minimizing a penalized cost function. Comprehensive surveys of this evolution can be found in [2] and [18].

From an algorithmic standpoint, dynamic programming (DP) has emerged as the canonical tool for solving optimal partitioning problems exactly. While classical DP incurs quadratic time and memory complexity, a substantial line of research has focused on pruning strategies to improve practical performance. Notable examples include PELT [10], functional pruning [16], and segment neighborhood methods [3]. Although highly effective under favorable conditions, these methods rely on assumptions about the cost structure or penalty function and may lose efficiency when segment costs are complex, constraints are added, or worst-case behavior is encountered.

Beyond exact DP-based approaches, a variety of heuristic and approximate methods have been proposed to address scalability concerns. Binary segmentation and its variants [17] [7] iteratively split the series using local test statistics and are computationally efficient, but they are known to suffer from suboptimal global behavior. Wild binary segmentation and related multiscale procedures improve detection power but still lack a global optimization perspective and provide limited control over structural constraints.

Sliding-window and online heuristics, often based on distance or local statistics, have been widely explored for detecting changes very shortly after its occurrence (low-latency detection, e.g., [8] [9]). While computationally efficient, such methods typically rely on local information and offer limited control over global segmentation structure.

Optimization-based heuristics form a smaller but growing subset of the CPD literature. Several authors have proposed mixed-integer programming formulations for changepoint detection, highlighting their flexibility and modeling power. However, due to the combinatorial nature of the resulting problems, these formulations are often limited to moderate instance sizes when solved to optimality.

To mitigate this issue, relaxation-based methods, warm-start strategies, and restricted search techniques have been explored, though often without a systematic heuristic framework.

Matheuristics offer a principled way to combine the strengths of these two worlds. By embedding mathematical programming models within heuristic search strategies, matheuristics aim to preserve global structure while avoiding exhaustive enumeration. Despite their success in other combinatorial optimization domains, their application to changepoint detection remains relatively underexplored. Existing approaches typically focus either on speeding up exact solvers or on designing problem-specific heuristics, with limited integration between the two.

The method proposed in this paper contributes to this emerging area by introducing a fore–n–back matheuristic that explicitly leverages an optimization-based formulation while maintaining heuristic flexibility and anytime performance. Unlike classical DP or purely algorithmic heuristics, the proposed approach is designed to accommodate complex cost structures and global constraints, positioning it as a complementary alternative within the broader CPD literature.

3 Fore-and-Back for changepoint analysis

Developed as an enhancement of traditional Beam Search (BS), a variant of standard tree search that limits the number of offsprings that are expanded at each iteration, the Fore-and-Back method (F&B, [4, 14]) aims to increase the overall effectiveness of the standard BS approach [11, 15, 5]. While Fore-and-back functions as an exact solution algorithm when provided with unlimited computational power, its primary intent is to serve as a heuristic. In this capacity, it prioritizes the rapid discovery of high-quality solutions over the rigorous pursuit of optimality proofs.

A defining feature of this approach is its ability to generate bounds on the cost required to complete partial solutions, even though it operates as a *primal-only method*. By calculating these bounds, the algorithm can identify and prune unpromising partial solutions before they are expanded, effectively narrowing the search space and increasing efficiency.

Beam search (BS) does not perform the exhaustive exploration typical of branch-and-bound algorithms. Instead, at each expansion step it maintains a restricted set of partial solutions, called the *beam* $\mathbf{T}$, from which at most δ solutions are retained, where δ is the *beam width*. These solutions are selected according to a ranking criterion that estimates the potential quality of their extensions, for instance by using bounds on the cost of their possible completions.

F&B extends the general beam search paradigm by alternating explorations in opposite expansion directions and by learning from partial solutions generated in previous iterations, which are stored in memory. These learned partial results are then exploited as a look-ahead mechanism to complete partial solutions when the search proceeds in the opposite direction. Consequently, the algorithm is

particularly effective for problems that admit a natural direction for constructing partial solutions, which can also be meaningfully reversed.

F&B leverages this structure by embedding it into an iterative heuristic framework that adopts a memory-based learning and look-ahead strategy, explicitly exploiting information learned during earlier search phases. The algorithm performs a partial exploration of the solution space through a sequence of beam-search-like trees of two types, referred to as forward and backward trees. Each node at level h in these trees corresponds to a partial solution consisting of h solution components.

At iteration t , a forward tree F^t is constructed if t is odd, whereas a backward tree B^t is generated if t is even. During the construction of a tree, each partial solution X is extended to a feasible complete solution by combining it with learned partial solutions produced in the previous iteration of the complementary tree. The cost of the resulting solution is then used to bound the quality of the best complete solution that can be obtained from X .

A list, denoted by L_h^t , is associated with each level h of the tree built at iteration t . The list contains δ nodes generated but not expanded at level h , where δ is an input control parameter equivalent to the likewise denoted parameter of Beam Search. All nodes so far expanded at level h in all trees at odd iterations are kept in set E^h for forward trees, while those expanded at level h in all trees at even iterations are kept in $\bar{E}^h$ for backward trees. The nodes in the lists L_h^t , $h = 1, \dots, n$, represent the memory of iteration t , that will be used to guide the exploration of the current tree and of the tree that will be explored in the following iteration $t + 1$.

The core idea of Fore-and-Back is to evaluate the completion cost of partial solutions stored at level h of the tree by means of the partial solutions stored in L_{n-h}^{t-1} in case of forward trees, or, analogously, the completion of partial solutions in L_{n-h}^t by means of solutions in L_h^{t-1} in case of backward trees.

Algorithm 1 presents a pseudocode for F&B. In it, $\mathbf{T}$ denotes alternatively the forward tree $\mathbf{F}$ or the backward tree $\mathbf{B}$, depending on the parity of the iteration counter, and T_h denotes the level h of the tree $\mathbf{T}$, be it forward or backward. The algorithm is controlled by three user-defined parameters: δ , the number of nodes expanded at each level of both forward and backward trees; $maxn$, the maximum total number of nodes to expand; $maxnodes$, the limit of the number of nodes of each tree.

In order to expand level h of a tree at iteration t , the algorithm computes the value $c(X)$ for each node $X \in T_h$ and builds the set $L_h^t \subseteq T_h$. The list is defined ordering T_h by increasing cost values and keeping in L_h^t only its δ nodes having the smallest cost, which will be further expanded.

A distinctive characteristic of the Fore-and-Back algorithm is that, at each level h of the tree generated at iteration t , it also records the value $\hat{c}_h^t$. This value corresponds to the minimum cost among the nodes in T_h that do not belong to L_h^t . It represents the smallest cost that is expected to be incurred when completing a complementary partial solution without relying on the partial solutions that contributed to the construction of L_h^t .

More details about the algorithm can be found in [14], a sample implementation of F&B applied to the GAP problem can be downloaded from [12].

4 Computational results

The F&B algorithm for the CPD problem was implemented inC++ and executed on a Windows 11 laptop equipped with an Intel Core Ultra 7 processor and 1 TB of RAM. Particular attention was devoted to the design of the data structures used to maintain the stages at each level of the search trees. These stages were implemented using the standard library container `std::map<int, int>`, which is realized as a *Red-Black* tree and therefore maintains its keys in sorted order. In this structure, the key corresponds to the number *num* of arcs used so far in the partial solution, while the associated value stores the minimum cost observed so far for that specific *num* and for all larger values up to the next existing key.

Benchmark instances. The instances in the test set were selected to be representative of a variety of application areas in which time series analysis is relevant. We leveraged benchmarks from the M forecasting competitions [1], including instances from the M3, M4, and M6 installments. Specifically, the M3 and M4 datasets include time series from microeconomics, industry, macroeconomics, finance, demographics, and an unspecified “other” category, while the M6 dataset consists of daily adjusted close prices of the competition assets. In addition, we complemented the test set with IoT time series collected from environmental sensors, as described in [13].

Benchmark codes. Different codes were used to benchmark the effectiveness of F&B under various application conditions. In the simplest setting, where the objective is solely to identify effective changepoints and the quality of a model is assumed to be additive over its constituent segments, the time points of the series can be mapped to the nodes of a graph, with segments represented as arcs. Under this formulation, the CPD problem reduces to finding a least-cost path from time 0 to the final time point in the resulting graph. This problem can be efficiently solved, for example, using the DAG-SHORTEST-PATHS (DSP) algorithm [6].

A commonly imposed constraint limits the maximum number of segments in the solution, specified as a parameter. Under this constraint, the problem remains easy to solve, as it requires only a minor adaptation of the Bellman–Ford shortest-path recursion [6], in which the number of dynamic programming stages is capped at the prescribed parameter value. We implemented this approach denoting it BF.

Finally, the state-of-the-art for CPD solution is represented by the *ruptures* library [18], a Python library for offline change point detection that provides a variety of methods for the analysis and segmentation of non-stationary signals. We include, as baselines, three widely used changepoint detection algorithms implemented in the *ruptures* library, namely PELT, dynamic programming (DYNP),

Algorithm 1: Algorithm Fore-and-Back

```

1 function Fore-and-Back( $\delta$ ,  $maxn$ ,  $maxnodes$ );
   Input :  $\delta$ , beam width,  $maxn$ , max total num of nodes,  $maxnodes$ , max
           num of nodes per tree
   Output: A feasible solution  $\mathbf{x}^*$  of value  $z_{best}$ 
2 initialize  $t = 0$ ,  $noimpr = 0$ ,  $nNodes = 0$ ,  $z_{best} = \infty$ ;
3 while  $nNodes \leq maxn$  do // Alternate forward and backward trees
4    $t = t + 1$ ;  $noimpr = noimpr + 1$ ;
5   let  $L_0^t = L_{n+1}^t = \{\emptyset\}$ ,  $ntNodes = 0$ ;
6   foreach level  $h = 1, \dots, n$  do
7     set  $T_h = \{\emptyset\}$ ; // generate the node set  $T_h$ 
8     if  $t$  is odd then
9       | set  $k = h$ ;  $k1 = h - 1$ ;
10    else
11      | set  $k = n - h + 1$ ;  $k1 = n - h + 2$ ;
12    end
13    foreach node  $X \in L_{k-1}^t$  do // Possible expansions
14      foreach component  $s \in S_k$  do
15        let  $X' = X \cup \{s\}$ ;
16        if  $X'$  meets conditions i) and ii) then
17          | set  $T_k = T_k \cup X'$ ;
18          |  $ntNodes = ntNodes + 1$ ;
19          |  $nNodes = nNodes + 1$ ;
20          if  $h = n$  and  $c(X') < z_{best}$  then // feasible solution
21            | set  $z_{best} = c(X')$ ,  $noimpr = 0$  and  $\mathbf{x}^* = X'$ ;
22          end
23        end
24      end
25      set  $E^k = E^k \cup X$ ; // actual node expansion
26    end
27    foreach node  $X \in T_k$  do // extract the subset  $L_k^t \in T_k$ 
28      if  $|T_k| \leq \delta$  then
29        | set  $L_k^t = T_k$ ;
30      else
31        | let  $L_k^t$  contain only the  $\delta$  least cost partial solutions of  $T_k$ ;
32      end
33    end
34    if  $ntNodes > maxnodes$  then break;
35  end
36  if  $noimpr = 2$  then stop; // sol. not improved for two iterations
37 end

```

and binary segmentation (BinSeg). These methods are representative of different algorithmic paradigms and trade-offs between optimality and computational efficiency.

Pruned Exact Linear Time (PELT) [10], is an exact method for optimal multiple changepoint detection under an additive cost formulation with a penalty on the number of changepoints. PELT exploits a pruning rule that discards suboptimal candidate changepoints during the dynamic programming recursion, yielding linear expected time complexity under mild assumptions on the cost function and penalty. PELT is often regarded as a state-of-the-art exact method for large-scale offline changepoint detection.

Dynamic programming (DYNP) computes the globally optimal segmentation for a fixed number of changepoints (or segments) by exhaustively exploring all possible partitions of the time series. This approach dates back to early work by Auger and Lawrence [3] and guarantees optimality, but its computational complexity is quadratic or worse in the length of the series.

Binary segmentation (BinSeg) is a greedy, recursive procedure that iteratively detects a single changepoint over the entire series and then applies the same procedure to the resulting subsegments. Classical binary segmentation was introduced by Scott and Knott [17] and remains popular due to its simplicity and low computational cost.

Table 1. Table captions should be placed above the tables.

Heading level	Example	Font size and style
Title (centered)	Lecture Notes	14 point, bold
1st-level heading	1 Introduction	12 point, bold
2nd-level heading	2.1 Printing Area	10 point, bold
3rd-level heading	Run-in Heading in Bold. Text follows	10 point, bold
4th-level heading	<i>Lowest Level Heading.</i> Text follows	10 point, italic

Please try to avoid rasterized images for line-art diagrams and schemas. Whenever possible, use vector graphics instead (see Fig. 1).

A cost function obtained by normalizing the residual sum of squares by the square root of the segment length does not satisfy the subadditivity condition required by PELT, since splitting a segment can increase the total cost. Consequently, the correctness of PELT pruning is not guaranteed for such a cost. In fact, any concave normalization in segment length (such as division by the square root of the length, as in RMSE or QRMSE) breaks PELT’s subadditivity assumption.

Example: a flat series with some noise: $n_1 = n_2 = 50$, $RSS_1 = RSS_2 = 50$ (unit variance noise) Then: One segment $C(1, 100) = \frac{100}{\sqrt{100}} = 10$, Two segments

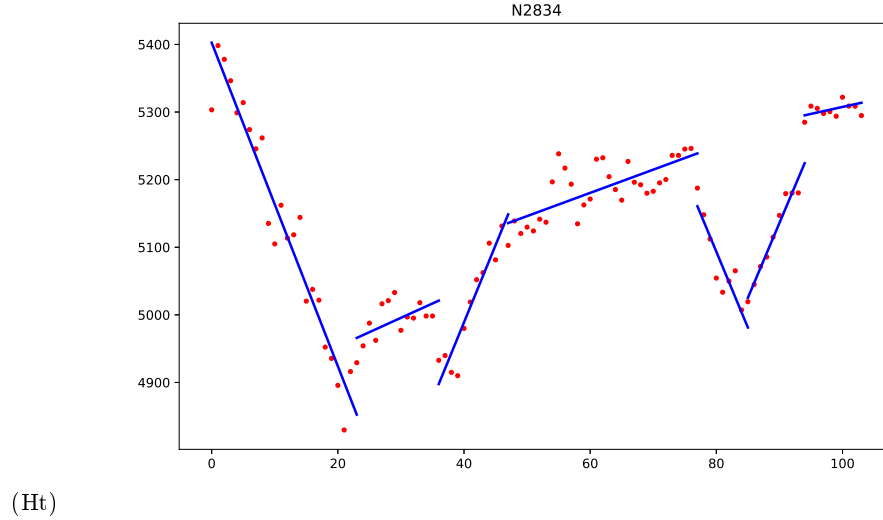


Fig. 1. A figure caption is always placed below the illustration. Please note that short captions are centered, while long ones are justified by the macro package automatically.

$$C(1, 50) + C(51, 100) = \frac{50}{\sqrt{50}} + \frac{50}{\sqrt{50}} = 7.07 + 7.07 = 14.14 \text{ So: } C(1, 100) < C(1, 50) + C(51, 100)$$

5 Conclusions

The conclusions

References

1. Makridakis competitions. https://en.wikipedia.org/wiki/Makridakis_Competitions, accessed: 5 January 2026
2. Aminikhanghahi, S., Cook, D.J.: A survey of methods for time series change point detection. *Knowledge and Information Systems* **51**, 339–367 (2017). <https://doi.org/10.1007/s10115-016-0987-z>
3. Auger, I.E., Lawrence, C.E.: Algorithms for the optimal identification of segment neighborhoods. *Bulletin of Mathematical Biology* **51**(1), 39–54 (1989)
4. Bartolini, E., Maniezzo, V., Mingozzi, A.: An adaptive memory-based approach based on partial enumeration. In: Maniezzo, V., Battiti, R., Watson, J.P. (eds.) *LION 2*, LNCS 5313, pp. 12–24. Springer (2008)
5. Blum, C.: Beam-ACO for simple assembly line balancing. *INFORMS Journal on Computing* **20**(4), 618–627 (2008)
6. Cormen, T.H., Leiserson, C.E., Rivest, R.L., Stein, C.: *Introduction to Algorithms*. MIT Press, 3rd edn. (2009)

7. Fischetti, M., Lodi, A., Salvagnin, D.: Just MIP it! In: Maniezzo, V., Stützle, T., Voß, S. (eds.) *Matheuristics, Annals of Information Systems*, vol. 10, pp. 1–20. Springer (2009)
8. Keogh, E.J., Chu, S., Hart, D.M., Pazzani, M.J.: An online algorithm for segmenting time series. In: *Proceedings of the 2001 IEEE International Conference on Data Mining*. pp. 289–296. IEEE, San Jose, CA, USA (2001). <https://doi.org/10.1109/ICDM.2001.989541>
9. Keogh, E.J., Chu, S., Hart, D.M., Pazzani, M.J.: Segmenting time series: A survey and novel approach. <https://api.semanticscholar.org/CorpusID:8365617> (2002), accessed 2025-12-01
10. Killick, R., Fearnhead, P., Eckley, I.A.: Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association* **107**(500), 1590–1598 (2012)
11. Lowerre, B.: The HARP speech recognition system. Ph.D. thesis, Carnegie Mellon University, Pittsburgh, PA (1976)
12. Maniezzo, V.: MatheuristicsGAP: Accompanying code for Matheuristics, algorithms and implementations. <https://github.com/maniezzo/MatheuristicsGAP> (2021), accessed: 5 January 2026
13. Maniezzo, V.: Extended set covering for time series segmentation. In: *Metaheuristics: 15th International Conference, MIC 2024, Lorient, France, June 4–7, 2024, Proceedings, Part I*. p. 193–199. Springer-Verlag, Berlin, Heidelberg (2024)
14. Maniezzo, V., Boschetti, M.A., Stützle, T.: Matheuristics: Algorithms and Implementations. *EURO Advanced Tutorials on Operations Research*, Springer (2021). <https://doi.org/10.1007/978-3-030-64219-4>
15. Natarajan, B.K.: On the use of beam search in exact inference. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **11**(8), 883–887 (1989)
16. Rigai, G.: A pruned dynamic programming algorithm to recover the optimal segmentation of a signal. *Journal de la Société Française de Statistique* **156**(4), 180–205 (2015)
17. Scott, A.J., Knott, M.: A cluster analysis method for grouping means in the analysis of variance. *Biometrics* **30**(3), 507–512 (1974)
18. Truong, C., Oudre, L., Vayatis, N.: Selective review of offline change point detection methods. *Signal Processing* **167**, 107299 (2020). <https://doi.org/10.1016/j.sigpro.2019.107299>